PROJECT REVIEW

MID REVIEW - INDIVIDUAL CONTRIBUTION (WEEK 1 TO WEEK 4)

COURSE CODE: 24ADI204

COURSE TITLE: DATA SCIENCE AND VISUALIZATION

Renewable Energy and Weather Analysis

SUBMITTED BY

V. NIDHISH (25BAD067)

Team No: 8

Team members: K S. Nagul Nisok (25BAD063), B. Nirmal (25BAD068), S. Nivesh (25BAD073)

MID REVIEW - SEPTEMBER 2026

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

KUMARAGURU COLLEGE OF TECHNOLOGY

(Autonomous Institution)

COIMBATORE - 641 049

INTRODUCTION:

This is my individual review for the mid review of our project, Renewable Energy and Weather Analysis (Weeks 1 to 4). The project looks at how daily weather changes solar and wind power at five places in Tamil Nadu: Coimbatore, Salem, Madurai, Tiruchirappalli and Erode. We work with a daily log of 510 records that has weather readings along with solar, wind, total renewable and demand figures.

My main work was **Week 2, Know Your Data**. I went through the structure and the summary numbers of the dataset, and that is where we first saw that the file only looks complete. The normal null check says nothing is missing, but the summary shows impossible minimum values of -999. I also worked on three other weeks. In Week 1 I loaded the dataset and checked it, in Week 3 I worked out the missing percentage and drew the missing-value heatmap, and in Week 4 I made the scatter plots. All the code in this review is taken from our team notebook (DSV Week 1 to Week 4).

Domain:

Renewable Energy / Environmental Data Analytics. Solar and wind power depend on the weather of the day. If we know which weather factors matter most, grid operators can plan better for days when power falls short.

Problem Statement:

Whenever a sensor fails, the file stores -999 instead of leaving the cell blank, and pandas reads it as a normal number. Because of this, the mean wind speed shows -2.71 km/h and the lowest temperature shows -999 degrees Celsius, which cannot be true. So the problem has two parts. First, find and handle the hidden -999 code, along with the 8 repeated rows found by the duplicate check and some very high values. Second, use EDA to find which weather columns are linked most strongly to solar, wind and total renewable power.

MY ROLE IN THE PROJECT:

Week	Phase	What I did
1	Team Formation and Data Hunting	Loaded the dataset with Pandas and checked its shape and columns.
2	Know Your Data	Ran and explained head, tail, info, dtypes and describe(). Wrote up the first-look findings for the First Impression Report.
3	The Cleaning Sprint	Worked out the missing percentage of every column and drew the missing-values heatmap.
4	EDA Deep Dive	Made the scatter plots and checked how strong each relationship is.

OBJECTIVES:

1. To load the dataset with Pandas and check its shape and columns.

2. To look at the data using head(), tail(), info(), dtypes and describe().

3. To read the summary numbers carefully and spot -999 as a hidden missing-value code.

4. To find the missing percentage of each column and show where the gaps are.

5. To draw scatter plots that show how weather columns relate to solar irradiance and total renewable energy.

6. To pass on findings that help with cleaning in Week 3 and with choosing columns for Weeks 5 and 6.

WEEK 1:

TEAM FORMATION AND DATA HUNTING

MY TASK:

Load the dataset with Pandas, make sure it opens properly, and check its shape and columns before any analysis starts.

ABOUT DATASET:

The data is our own simulated daily log with 510 records from five places in Tamil Nadu. Salem has the most rows (115). In our notebook the file is named **renewable_energy_weather.csv.xls** and it is loaded with **pd.read_csv** (the project documents call it renewable_energy_weather.csv). There are 14 columns:

Group	Columns	Type
Identifiers	Date, Location	object (text)
Weather	Temperature_C, Humidity_pct, Wind_Speed_kmh, Solar_Irradiance_Wm2, Cloud_Cover_pct, Rainfall_mm, Air_Pressure_hPa	float64
Weather label	Weather_Condition	object (text)
Energy	Solar_Energy_kWh, Wind_Energy_kWh, Total_Renewable_Energy_kWh, Energy_Demand_kWh	float64

Loading the dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import zscore
df = pd.read_csv("renewable_energy_weather.csv.xls")
print("Dataset loaded successfully!")
print("Shape:", df.shape)
display(df.head())
```

Output:

```
Dataset loaded successfully!
Shape: (510, 14)
```

display(df.head()) prints the first five rows. They are shown in Week 2, Step 1.

Checking the rows, columns and column means

```
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
print("\nColumns:")
print(df.columns.tolist())
print("\nNumerical Mean:")
print(df.select_dtypes(include=np.number).mean())
```

Output:

```
Rows: 510
Columns: 14

Columns:
['Date', 'Location', 'Temperature_C', 'Humidity_pct', 'Wind_Speed_kmh',
 'Solar_Irradiance_Wm2', 'Cloud_Cover_pct', 'Rainfall_mm', 'Air_Pressure_hPa',
 'Solar_Energy_kWh', 'Wind_Energy_kWh', 'Total_Renewable_Energy_kWh',
 'Energy_Demand_kWh', 'Weather_Condition']

Numerical Mean:
Temperature_C                  13.109804
Humidity_pct                   51.183902
Wind_Speed_kmh                 -2.713314
Solar_Irradiance_Wm2          533.891941
Cloud_Cover_pct                23.643412
Rainfall_mm                    -8.103020
Air_Pressure_hPa              990.603902
Solar_Energy_kWh                3.903098
Wind_Energy_kWh               116.937137
Total_Renewable_Energy_kWh    140.965451
Energy_Demand_kWh             197.571529
dtype: float64
```

INTERPRETATION:

The dataset opens without any problem and has 510 rows and 14 columns, which matches the description. The column means are where the trouble first shows. Wind_Speed_kmh has a mean of -2.713314 km/h and Rainfall_mm has a mean of -8.103020 mm. Neither of these can be negative in real life. I passed this on to Week 2 for a closer look.

MY CONTRIBUTION:

Task	Result
Loaded the CSV with Pandas	Dataset opened without errors
Checked shape, columns and means	510 rows x 14 columns; negative means for wind speed and rainfall

WEEK 2:

KNOW YOUR DATA

OBJECTIVE:

The aim is to understand the dataset before doing anything else: its size, the type of each column, some sample rows and a statistical summary. This is the week where we first noticed the -999 problem, so it decided what we did in the cleaning week.

USED PACKAGES AND METHODS:

Packages: pandas, numpy
Methods: head(), tail(), info(), dtypes, describe()

Step 1: Looking at the first and last rows

```
display(df.head())
display(df.tail())
```

Output of head() (first 5 rows; only the first 5 of 14 columns are shown):

	Date	Location	Temperature_C	Humidity_pct	Wind_Speed_kmh
0	2025-01-01	Salem	29.06	67.20	11.79
1	2025-01-02	Tiruchirappalli	24.42	74.57	9.20
2	2025-01-03	Madurai	31.03	67.97	8.32
3	2025-01-04	Tiruchirappalli	27.79	79.04	5.96
4	2025-01-05	Tiruchirappalli	26.86	69.68	14.02

Output of tail() (last 5 rows):

	Date	Location	Temperature_C	Humidity_pct	Wind_Speed_kmh
505	2025-06-05	Coimbatore	28.54	53.26	14.49
506	2025-09-07	Madurai	20.20	67.47	8.93
507	2025-08-06	Salem	29.86	84.20	8.55
508	2025-08-10	Coimbatore	25.42	71.70	8.62
509	2025-10-21	Salem	23.38	80.87	14.76

The first five rows are consecutive days starting from 2025-01-01. The last five rows are not in date order: their dates are 2025-06-05, 2025-09-07, 2025-08-06, 2025-08-10 and 2025-10-21. head() and tail() only show where rows sit in the file, so they cannot tell us when or why the rows are placed this way. This pattern, together with the 10 excess date occurrences that describe() shows in Step 3, is worth checking further.

Step 2: File structure with info()

```
df.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 510 entries, 0 to 509
Data columns (total 14 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Date                        510 non-null    object
 1   Location                    510 non-null    object
 2   Temperature_C               510 non-null    float64
 3   Humidity_pct                510 non-null    float64
 4   Wind_Speed_kmh              510 non-null    float64
 5   Solar_Irradiance_Wm2        510 non-null    float64
 6   Cloud_Cover_pct             510 non-null    float64
 7   Rainfall_mm                 510 non-null    float64
 8   Air_Pressure_hPa            510 non-null    float64
 9   Solar_Energy_kWh            510 non-null    float64
 10  Wind_Energy_kWh             510 non-null    float64
 11  Total_Renewable_Energy_kWh  510 non-null    float64
 12  Energy_Demand_kWh           510 non-null    float64
 13  Weather_Condition           510 non-null    object
dtypes: float64(11), object(3)
memory usage: 55.9+ KB
```

info() shows 510 entries, 14 columns and 510 non-null values in every column. The file uses about 55.9 KB. Nothing looks missing here, and that is the trap.

Step 3: Data types and the statistical summary

```
print("Data Types:")
print(df.dtypes)
print("\nStatistical Summary:")
display(df.describe(include="all"))
```

Output:

```
Data Types:
Date                        object
Location                    object
Temperature_C               float64
Humidity_pct                float64
Wind_Speed_kmh              float64
Solar_Irradiance_Wm2        float64
Cloud_Cover_pct             float64
Rainfall_mm                 float64
Air_Pressure_hPa            float64
Solar_Energy_kWh            float64
Wind_Energy_kWh             float64
Total_Renewable_Energy_kWh  float64
Energy_Demand_kWh           float64
Weather_Condition           object
dtype: object

Statistical Summary:
              Date  Location  Temperature_C  Humidity_pct  Wind_Speed_kmh
count          510       510     510.000000    510.000000      510.000000
unique         500         5            NaN           NaN             NaN
top     2026-02-19     Salem            NaN           NaN             NaN
freq             2       115            NaN           NaN             NaN
mean           NaN       NaN      13.109804     51.183902       -2.713314
std            NaN       NaN     127.979791    132.977233      117.785407
min            NaN       NaN    -999.000000   -999.000000     -999.000000
25%            NaN       NaN      25.892500     61.492500        8.112500
50%            NaN       NaN      29.380000     67.425000       10.575000
75%            NaN       NaN      32.467500     73.667500       13.002500
max            NaN       NaN      48.000000     94.920000       80.000000
```

The summary table above shows the first five of the 14 columns. Here is what I noticed in it:

- **Date** has 510 values but only 500 are unique, which means there are 10 excess (repeated) date occurrences. It is also stored as text (object), so it has to be converted to a date before we can use the month or season.
- **Location** has 5 places, and Salem is the most common with 115 rows.
- Temperature_C, Humidity_pct and Wind_Speed_kmh all have a minimum of -999.00. Their standard deviations are huge (127.98, 132.98 and 117.79). The -999 code appears in multiple numeric columns (nine in total, as Week 3 shows) and stands for hidden missing values. These three columns show the effect clearly in the part of the summary printed here.
- The 25%, 50% and 75% values look normal, for example 25.89, 29.38 and 32.47 for temperature. The mean (13.11) is far away from the median (29.38). This suggests that only a small share of the rows is affected.

Every one of these columns has a limit in real life, so the minimum row of describe() is a quick way to spot wrong values. All three raw minimums break the limit:

Column	Possible range in real life	Raw minimum	Verdict
Temperature_C	Cannot go below -273.15 (in practice much higher)	-999.00	Impossible
Humidity_pct	0 to 100	-999.00	Impossible
Wind_Speed_kmh	0 or more	-999.00	Impossible

Step 4: Confirming that -999 is the reason

In Week 3 the missing percentages showed 8 hidden gaps in Temperature_C and 7 in Wind_Speed_kmh (1.57% and 1.37% of 510 rows). If those cells really hold -999, I should be able to rebuild the raw numbers from the cleaned ones. It works:

Check	Calculation	Rebuilt	Observed (raw)
Temperature_C mean	(502 x 29.24 + 8 x -999) / 510	13.11	13.109804
Temperature_C std	sqrt(p(1-p)) x (29.24 + 999), p = 8/510	about 127.8	127.979791
Wind_Speed_kmh mean	(503 x 11.14 + 7 x -999) / 510	about -2.72	-2.713314

The numbers match, which supports the idea that -999 is a missing-value code and not a real reading.

The team's null and duplicate checks

```
print("Missing Values:")
print(df.isnull().sum())
print("\nDuplicate Rows:", df.duplicated().sum())
```

Output:

```
Missing Values:
Date                        0
Location                    0
Temperature_C               0
Humidity_pct                0
Wind_Speed_kmh              0
Solar_Irradiance_Wm2        0
Cloud_Cover_pct             0
Rainfall_mm                 0
Air_Pressure_hPa            0
Solar_Energy_kWh            0
Wind_Energy_kWh             0
Total_Renewable_Energy_kWh  0
Energy_Demand_kWh           0
Weather_Condition           0
dtype: int64

Duplicate Rows: 8
```

B. Nirmal ran this check. I have added it here because it shows why my summary mattered. isnull() finds nothing wrong because pandas treats -999 as a normal number, so only describe() shows the problem. The check also finds 8 repeated rows. The Date column has 10 excess date occurrences, and 8 of them can come from these exact copies. So the other 2 are rows that share a date but are not exact copies. That needs a look before the repeated rows are removed.

First Impression Report

```
print("FIRST IMPRESSION REPORT")
print("======================")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
print("Total Missing Values:", df.isnull().sum().sum())
print("Duplicate Rows:", df.duplicated().sum())
print("\nData Types:")
print(df.dtypes)
```

Output:

```
FIRST IMPRESSION REPORT
======================
Rows: 510
Columns: 14
Total Missing Values: 0
Duplicate Rows: 8
```

After this it prints the same data types list as in Step 3.

How much the -999 code changed the column means

Comparing the raw means from Week 1 with the means after the Week 3 cleaning shows how much the -999 code spoiled the first summary. Total_Renewable_Energy_kWh had no -999 values, so its mean stays the same.

Column	Mean before cleaning	Mean after cleaning
Temperature_C	13.11	29.24
Humidity_pct	51.18	67.91
Wind_Speed_kmh	-2.71	11.14
Solar_Irradiance_Wm2	533.89	555.81
Cloud_Cover_pct	23.64	35.75
Rainfall_mm	-8.10	3.66
Air_Pressure_hPa	990.60	1010.31
Solar_Energy_kWh	3.90	13.84
Total_Renewable_Energy_kWh	140.97	140.97 (no change)

INTERPRETATION:

The dataset has 510 daily records and 14 columns: 11 numeric columns and 3 text columns. The isnull() check says there are no missing values, which looks fine, but describe() shows something else. Many columns have an impossible minimum of -999.00 (the -999 code turns out to be in nine numeric columns), and the std of Temperature_C jumps to 127.98. Eight -999 values among 510 normal readings give almost exactly this figure. The quartiles still look normal, so only a small part of the rows is affected. The Date column has 10 excess date occurrences (510 values, 500 unique), and the duplicate check finds 8 repeated rows. So the dataset only looks complete at first sight. The -999 code had to be changed to a proper missing value before any number could be trusted, and that is what the team did in Week 3.

MY CONTRIBUTION:

Task	Result
Looked at the data with head(), tail() and info()	510 entries, 14 columns, 55.9+ KB; the last 5 rows are not in date order
Checked column types with dtypes	11 float64 and 3 object columns; Date is stored as text
Ran and explained describe()	Found the impossible -999 minimums and the std of 127.98
Noticed the repeated date occurrences	500 unique dates in 510 rows (10 excess occurrences)
Tested the -999 idea with a calculation	Rebuilt the raw means and std from the cleaned values
Wrote the first-look findings	Used in the First Impression Report and the Week 3 plan

WEEK 3:

THE CLEANING SPRINT

MY TASK:

K S. Nagul Nisok first changed -999 to NaN (the first code block below). I have included it because my missing percentages depend on it. After that I found out how much of each column is really missing and drew a heatmap to see where the gaps are.

Changing -999 to NaN (done by K S. Nagul Nisok)

```
# The dataset uses -999 as a missing-value code
df = df.replace(-999, np.nan)
print("Missing values after converting -999 to NaN:")
print(df.isnull().sum())
```

Output:

```
Missing values after converting -999 to NaN:
Date                         0
Location                     0
Temperature_C                8
Humidity_pct                 8
Wind_Speed_kmh               7
Solar_Irradiance_Wm2         7
Cloud_Cover_pct              6
Rainfall_mm                  6
Air_Pressure_hPa             5
Solar_Energy_kWh             5
Wind_Energy_kWh              5
Total_Renewable_Energy_kWh   0
Energy_Demand_kWh            0
Weather_Condition            0
dtype: int64
```

Finding the missing percentage of each column

```
missing_percent = (df.isnull().sum() / len(df)) * 100
display(missing_percent.to_frame("Missing Percentage"))
```

Output:

	Missing Percentage
Date	0.000000
Location	0.000000
Temperature_C	1.568627
Humidity_pct	1.568627
Wind_Speed_kmh	1.372549
Solar_Irradiance_Wm2	1.372549
Cloud_Cover_pct	1.176471
Rainfall_mm	1.176471
Air_Pressure_hPa	0.980392
Solar_Energy_kWh	0.980392
Wind_Energy_kWh	0.980392
Total_Renewable_Energy_kWh	0.000000
Energy_Demand_kWh	0.000000
Weather_Condition	0.000000

Showing where the missing values are

```
plt.figure(figsize=(12,5))
sns.heatmap(df.isnull(), cbar=False)
plt.title("Missing Values Visualization")
plt.show()
```

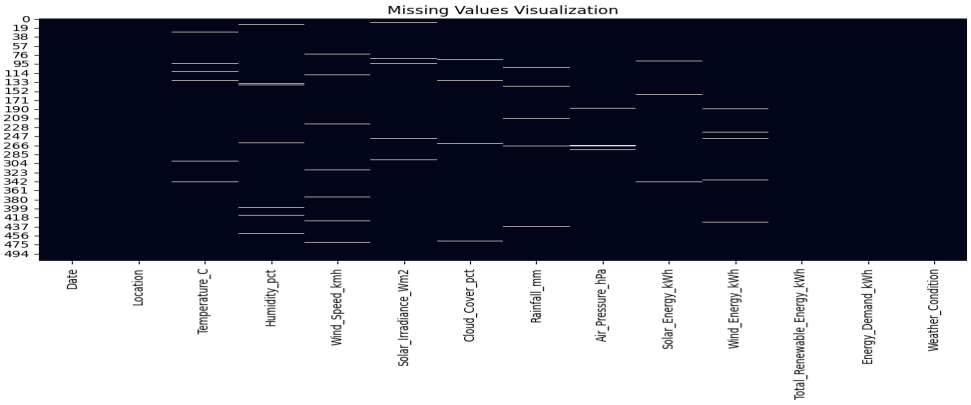


Figure 1: Missing-values heatmap. Each thin light line is one missing cell.

INTERPRETATION:

Nine numeric columns have gaps, and each one loses only about 1 to 1.6 percent of its rows. That is 8 rows for Temperature_C and Humidity_pct, 7 for Wind_Speed_kmh and Solar_Irradiance_Wm2, 6 for Cloud_Cover_pct and Rainfall_mm, and 5 each for Air_Pressure_hPa, Solar_Energy_kWh and Wind_Energy_kWh. That makes 57 gaps in total. On the heatmap the gaps show up as thin separate lines spread over the rows, not as long blocks, so no long stretch of days is missing. The amount is small, so filling each gap with the column median is a sensible choice. Total_Renewable_Energy_kWh and Energy_Demand_kWh have no gaps.

One limit of this heatmap: it only shows real NaN values. The Weather_Condition column also had "-999" written as text. A numeric replace does not touch it and the heatmap cannot show it, so that column needs a separate check (see Future Work).

MY CONTRIBUTION:

Task	Result
Found the missing percentage of each column	0.98% to 1.57% in nine columns, 57 gaps in total
Drew the missing-values heatmap	Gaps are scattered, not clustered

WEEK 4:

EDA DEEP DIVE

MY TASK:

Using the cleaned data, I made the scatter plots that compare weather columns with solar irradiance and with total renewable energy, and checked how strong each link is.

Temperature against solar irradiance

```
plt.figure(figsize=(8,5))
sns.scatterplot(data=df, x="Temperature_C", y="Solar_Irradiance_Wm2")
plt.title("Temperature vs Solar Irradiance")
plt.xlabel("Temperature_C")
plt.ylabel("Solar_Irradiance_Wm2")
plt.show()
```

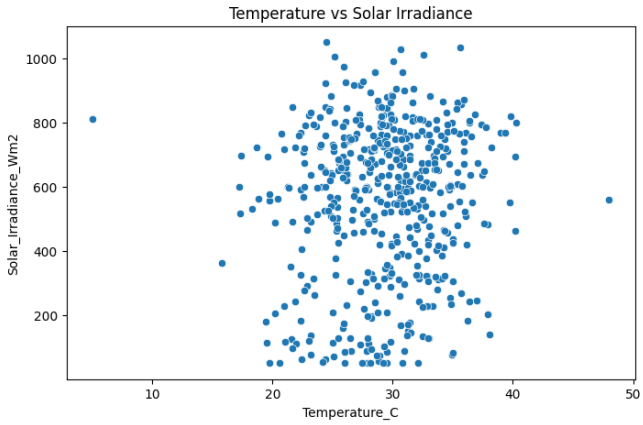


Figure 2: Temperature vs Solar Irradiance (r = 0.09).

The points look like a shapeless cloud. At 20 or at 40 degrees C, solar irradiance can be anything from about 50 to over 1000 W/m2, so temperature has only a weak link with sunlight (r = 0.09). Two lone points at about 5 and 48 degrees C match the two Z-score outliers we found in Temperature_C in Week 3.

Humidity against solar irradiance

```
plt.figure(figsize=(8,5))
sns.scatterplot(data=df, x="Humidity_pct", y="Solar_Irradiance_Wm2")
plt.title("Humidity vs Solar Irradiance")
plt.xlabel("Humidity_pct")
plt.ylabel("Solar_Irradiance_Wm2")
plt.show()
```

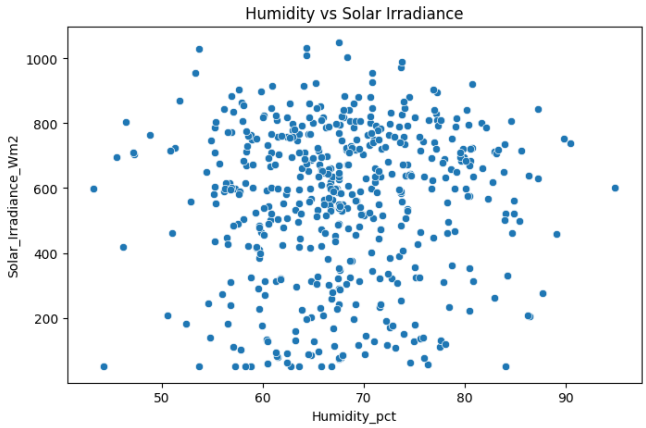


Figure 3: Humidity vs Solar Irradiance (r = 0.03).

This is another shapeless cloud. Whether humidity is 50 or 90 percent, sunlight can be low or high, so there is almost no link (r = 0.03).

Wind speed against total renewable energy

```
plt.figure(figsize=(8,5))
sns.scatterplot(data=df, x="Wind_Speed_kmh", y="Total_Renewable_Energy_kWh")
plt.title("Wind Speed vs Total Renewable Energy")
plt.xlabel("Wind_Speed_kmh")
plt.ylabel("Total_Renewable_Energy_kWh")
plt.show()
```

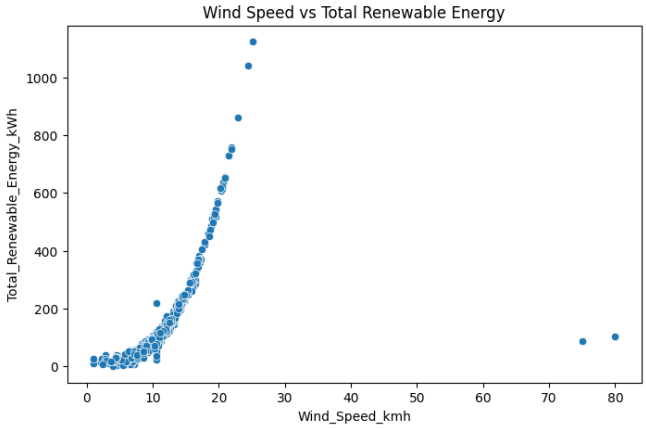


Figure 4: Wind Speed vs Total Renewable Energy (r = 0.60).

Total energy rises quickly as wind speed goes up, but in a curve and not a straight line. It stays low up to about 10 km/h, then climbs steeply: a few hundred kWh at 15 to 20 km/h and over 1000 kWh at 25 km/h. This fits how wind power works, since it grows roughly with the cube of wind speed. The correlation value r = 0.60 only measures straight-line links, so it makes the relationship look weaker than it really is.

INTERPRETATION:

Relationship	r	What the plot shows
Temperature vs solar irradiance	0.09	Very weak association
Humidity vs solar irradiance	0.03	Almost no link
Wind speed vs total renewable energy	0.60	Relatively strong but curved, so r looks lower

The r values come from the team's correlation matrix.

Temperature and humidity show very weak relationships with solar irradiance, so other weather factors should be investigated. The team's correlation matrix (made by B. Nirmal) shows a strong negative value between cloud cover and solar irradiance (r = -0.86), so cloud cover should be looked at further. Wind speed shows a relatively strong positive association with total renewable energy and is a promising variable for further modelling. Since the link is curved, a transformed or non-linear approach may work better than a simple straight line.

Something to check in Week 5

Two points in Figure 4 are far to the right, at about 75 and 80 km/h, but they give only about 90 to 105 kWh. That is much lower than days with 20 to 25 km/h. They are the same two Z-score outliers in Wind_Speed_kmh. The team kept them as possible extreme wind days, but the low output makes me think they could be days when the turbines were shut down, or just recording errors. They also pull r down. I think we should check them before modelling.

MY CONTRIBUTION:

Task	Result
Made the three scatter plots	Temperature and humidity vs solar irradiance; wind speed vs total energy
Checked the strength of each link	r = 0.09 (weak), r = 0.03 (none), r = 0.60 (relatively strong, curved)
Noticed the extreme points	Matched them with the Week 3 Z-score outliers and marked them for review

CHALLENGES I FACED:

1. **The null check misled me.** isnull() showed 0 missing values, so the file looked clean. Only the minimum row of describe() showed the problem.

2. **Mixed data types.** Weather_Condition is a text column and it also had "-999" written as a word, so a numeric replace cannot catch it.

3. **Date is stored as text.** dtypes showed it as object, so it has to be converted before I can take the month or season from it.

4. **Repeated dates.** Date has 500 unique values in 510 rows, which means 10 excess date occurrences, but the duplicate check finds only 8 repeated rows. So the repeated records are not all exact copies.

5. **A curved relationship.** Wind speed and total energy are clearly related, but r = 0.60 makes the link look weaker.

6. **Real or faulty extremes.** Deciding whether values like 75 to 80 km/h wind were real events or errors needed careful thinking.

WHAT I LEARNED:

- A clean-looking summary does not prove the data is clean. Always compare describe() with real-world limits.
- Codes like -999 pass the normal null check. I could confirm it by rebuilding the statistics from the cleaned values.
- When the mean is far from the median, a few wrong values can be the reason.
- Correlation only measures straight-line links. Scatter plots show curves that r hides.
- A missing-value heatmap shows only real NaN values.

FUTURE WORK:

- Look at the 10 excess date occurrences, then remove the 8 repeated rows found by the duplicate check.
- Check Weather_Condition for the text "-999" before turning it into numbers.
- Change Date to a datetime and take out the month and season.
- Check the two extreme wind speed points (75 and 80 km/h) before modelling.
- Try a rank-based correlation, or transform wind speed, to capture its curved link with total energy.
- Make new columns such as a clear-sky index and a heat-humidity index (Weeks 5 and 6), then build simple models for Total_Renewable_Energy_kWh and Energy_Demand_kWh.

GitHub repository (from the mid-review presentation): nidhish4616-collab / DSV_Week1_to_Week4_Renewable_Enegry

REFERENCES:

1. Pandas Development Team. pandas documentation. https://pandas.pydata.org/docs/

2. Harris, C. R. et al. NumPy reference. https://numpy.org/doc/stable/

3. Waskom, M. seaborn: statistical data visualization. https://seaborn.pydata.org/

4. Hunter, J. D. Matplotlib documentation. https://matplotlib.org/stable/

5. Virtanen, P. et al. SciPy - scipy.stats.zscore. https://docs.scipy.org/doc/scipy/

6. Tukey, J. W. Exploratory Data Analysis. Addison-Wesley, 1977 (IQR and boxplot rules).

7. DSV Rule Book, 24ADI204 Data Science and Visualization - Weeks 1 to 4.